

UTTEC 바이브코딩 — 시스템 흐름 가이드

이 문서는 UTTEC 바이브코딩 교육 플랫폼이 **어떻게 동작하는지** 처음 접하는 사람도 이해할 수 있도록 설명합니다. 기술적 배경지식이 없어도 읽을 수 있습니다.

1. 한 줄 요약

"빨간 LED 깜빡여줘" 라고 말하면, 30초 뒤에 진짜 LED가 깜빡인다.

이것이 UTTEC 바이브코딩의 전부입니다. 한국어로 원하는 동작을 적으면, **AI가 코드를 만들고, 서버가 프로그램을 만들고, 무선으로 보드에 보내서** 실제로 동작합니다.

2. 등장인물 소개

이 시스템에는 4명의 "등장인물"이 있습니다.



등장인물	역할	비유
사용자 (나)	원하는 동작을 한국어로 입력	손님이 주문하는 것
AI (Claude)	한국어를 컴퓨터 코드로 번역	통역사 (한국어 → C언어)
빌드 서버	코드를 보드가 이해하는 프로그램으로 변환	공장에서 제품을 만드는 것
ESP32 보드	프로그램을 받아서 실제로 LED, 부저 등을 동작	명령을 실행하는 로봇

3. 전체 흐름 — 피자 주문에 비유하기

바이브코딩의 동작 과정은 **피자 주문**과 비슷합니다.

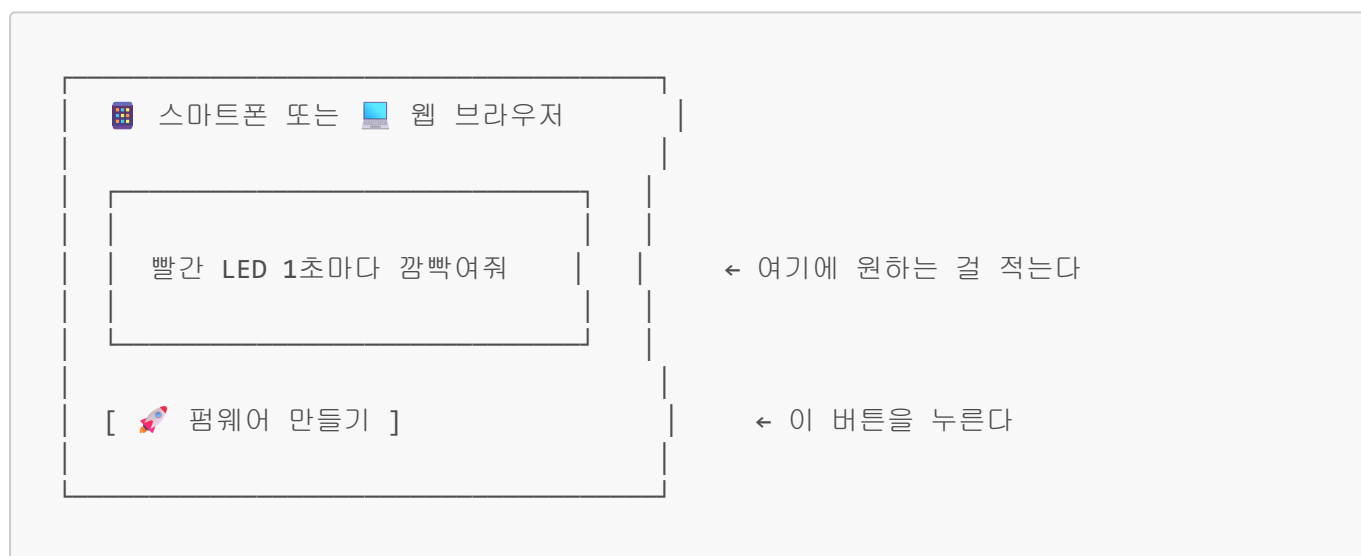
피자 주문

바이브코딩



4. 단계별 상세 설명

Step 1. 프롬프트 입력 (내가 하는 일)



프롬프트(Prompt) = AI에게 보내는 주문서

- 한국어로 적으면 됩니다
- 구체적으로 적을수록 좋습니다
- 예: "LED 켜줘" (O) → "빨간 LED 0.5초마다 3번 깜빡이고 멈춰줘" (더 좋음!)

접속 방법 2가지:

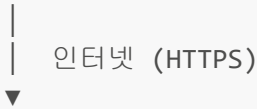
방법	도구	주소
웹 브라우저	Chrome	https://uttec-ai.duckdns.org/firmware
스마트폰 앱	Flutter APK	앱 설치 후 실행

Step 2. AI가 코드를 만든다 (약 8초)

버튼을 누르면, 내 주문이 인터넷을 통해 **클라우드 서버**로 전달됩니다. 서버에서는 ****AI(Claude)****가 내 한국어를 읽고, 컴퓨터 코드(C언어)를 만들어줍니다.

내가 적은 것:

"빨간 LED 1초마다 깜빡여줘"



AI가 만든 코드:

```

void ledTask(void *param) {
    while (1) {
        digitalWrite(25, LOW); // 켜
        delay(500);
        digitalWrite(25, HIGH); // 꺼
        delay(500);
    }
}

void setup() {
    Serial.begin(115200);
    initHardware();
    initBLE();
    xTaskCreate(ledTask, ...);
}

void loop() {
    delay(10000);
}
  
```

Q: 왜 직접 코드를 안 짜도 되나요? A: AI가 이미 ESP32 보드의 부품(LED, 부저, OLED 등)을 모두 알고 있기 때문입니다. "빨간 LED"라고 하면 AI가 "아, GPIO 25번 핀이구나!"라고 알아서 판단합니다.

Q: AI는 어떻게 우리 보드를 알고 있나요? A: 서버에 *****시스템 프롬프트*****라는 설명서가 미리 들어있습니다. 거기에 "빨간 LED는 25번 핀, 부저는 14번 핀..." 이런 정보가 모두 적혀 있어서 AI가 매번 그 정보를 읽고 코드를 만듭니다.

Step 3. 서버가 프로그램을 만든다 (약 25초)

AI가 만든 코드는 아직 "텍스트"일 뿐입니다. ESP32 보드는 텍스트를 읽을 수 없고, ****기계어(0과 1)****만 이해합니다.

그래서 서버가 코드를 기계어로 번역하는 과정을 거칩니다. 이것을 **"빌드(Build)"** 또는 *****컴파일(Compile)*****이라고 합니다.

AI가 만든 코드 (사람이 읽을 수 있는 글)

```
digitalWrite(25, LOW);
delay(500);
...
```

빌드 (컴파일)
= 번역 작업

기계어 파일 (보드가 읽을 수 있는 숫자)

```
01001000 11010010 10110001
00011010 01100111 11001010
... (약 650KB, 65만 글자)
```

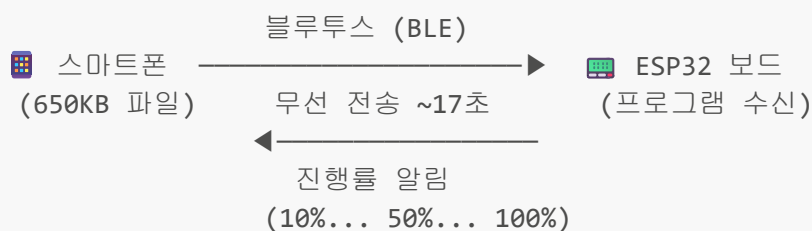
firmware.bin 파일

이 과정을 **Arduino-CLI**라는 도구가 수행합니다.

비유: 한국어 소설(코드)을 외국어(기계어)로 번역하는 것과 같습니다. 번역가(컴파일러)가 한 글자씩 꼼꼼히 번역하기 때문에 시간이 좀 걸립니다 (~25초).

Step 4. 무선으로 보드에 전송한다 (약 17초)

완성된 프로그램(firmware.bin)을 스마트폰/PC가 다운로드 받고, **블루투스(BLE)**를 통해 ESP32 보드에 무선으로 보냅니다.



이 과정을 **OTA(Over The Air)**라고 부릅니다. "공중으로(무선으로) 전달한다"는 뜻입니다.

Q: 왜 USB 케이블로 안 보내나요? A: 교실에서 30명 학생이 동시에 사용할 때, 각자 USB 케이블을 연결하는 것보다 블루투스로 무선 전송하는 것이 훨씬 편하기 때문입니다.

전송 과정 (자세히):

1. 스마트폰이 보드를 찾는다
2. 블루투스로 연결한다
3. "지금부터 프로그램 보낼게"
4. 조각조각 나눠서 보낸다



5. "다 보냈어!"  →  OTA 완료 신호
6. 보드가 새 프로그램으로 재시작   재부팅!

Step 5. 보드가 동작한다!

보드가 새 프로그램을 받으면 자동으로 재부팅하고, 바로 동작을 시작합니다.

ESP32 보드

-  빨간 LED: 깜빡깜빡!
 -  OLED: "UTTEC Firmware"
 -  부저: (조용히 대기중)
 -  센서: 26.3°C, 45%
- ← 내가 원하는 동작!

5. 전체 흐름 한눈에 보기

사용자

클라우드 서버

보드

① "빨간 LED 깜빡여줘"

↓ 인터넷 (HTTPS)

② AI가 코드 생성 (~8초)

"digitalWrite(25, LOW)..."

③ 빌드 = 기계어 번역 (~25초)

firmware.bin (650KB)

↓ 인터넷 (다운로드)

.bin 파일 수신

↓ 블루투스 (BLE OTA, ~17초)

④ 프로그램 수신

⑤ 재부팅 → 실행!

● 깜빡깜빡!

총 소요 시간: 약 35~70초 (프롬프트 입력부터 LED 깜빡이기까지)

6. 우리 보드에 달린 부품들

ESP32 보드에는 AI로 제어할 수 있는 여러 부품이 달려 있습니다.

ESP32 보드 (앞면)

● 빨간 LED

(25번 핀)

● 노란 LED

(26번 핀)

● 파란 LED

(27번 핀)

🔔 부저 (삐!)

(14번 핀)

🎵 멜로디 부저

(33번 핀)

📺 OLED 화면 (128x64 픽셀)

Hello World!

Temp: 26.3C

🌡️ 온습도 센서 (AHT20)

● 스위치 (32번 핀)

[USB 포트]

부품	할 수 있는 것	프롬프트 예시
● ● ● LED 3개	켜기, 끄기, 깜빡이기, 패턴 만들기	"신호등 만들어줘"
🔔 부저 (BEEP)	삐! 소리 내기	"부저 3번 울려줘"
🎵 멜로디 부저	도레미 음계, 동요 연주	"학교종 멜로디 연주해줘"
📺 OLED 화면	글자, 숫자, 그래프 표시	"OLED에 내 이름 보여줘"
🌡️ 온습도 센서	온도/습도 측정	"지금 온도 알려줘"
● 스위치	버튼 누름 감지	"스위치 누르면 LED 켜줘"

7. 웹앱 화면 구성

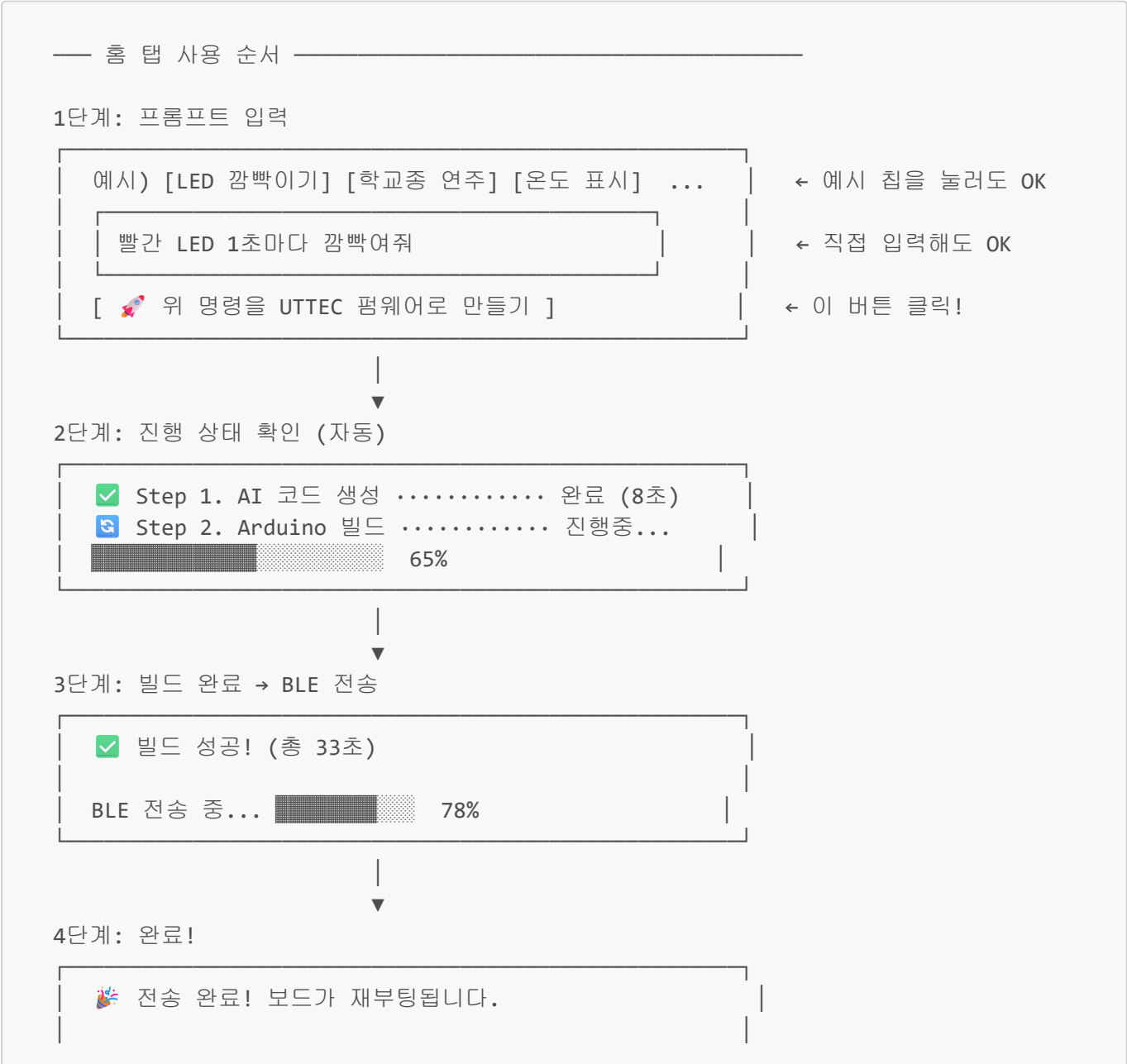
웹 브라우저로 접속하면 아래와 같은 5개의 탭이 있습니다.





탭	이름	하는 일
홈	메인 화면	프롬프트를 입력하고, 펌웨어를 만들어서 보드에 보내는 곳
HW	하드웨어 학습	보드에 달린 부품(LED, 부저, 센서 등)의 설명을 볼 수 있는 곳
SW	소프트웨어 학습	C언어, GPIO 등 프로그래밍 기초를 배울 수 있는 곳
패드	직접 제어	블루투스로 연결해서 LED, 부저를 버튼으로 바로 제어하는 곳
설정	환경 설정	서버 주소, BLE 기기 이름 등을 설정하는 곳

홈 탭 — 가장 많이 쓰는 화면



 생성된 코드 학습하기
 코드와 설명 보기 (접기/펼치기)

← 코드도 볼 수 있음

🎮 패드 탭 — 실시간 제어

패드 탭에서는 프롬프트 없이, **버튼을 눌러서 바로** 보드를 제어합니다. (블루투스로 연결되어 있어야 합니다)

```

LED 제어
[  빨강 ] [  노랑 ] [  파랑 ] [ 전체 끄기 ]

피아노
[도] [레] [미] [파] [솔] [라] [시] [도']

센서
[  온도 읽기 ] [  OLED에 글쓰기 ]
  
```

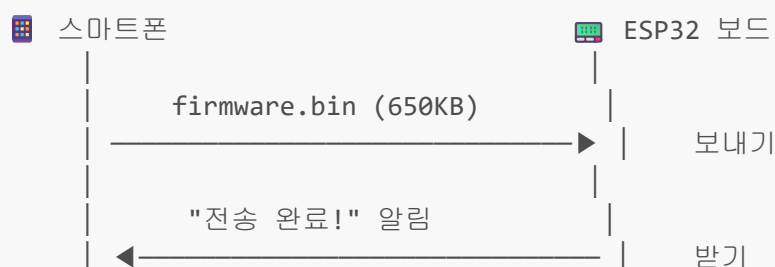
8. 통신 방식 이해하기

이 시스템에서는 **2가지 통신**이 사용됩니다.

통신 1: 인터넷 (HTTPS) — 스마트폰 ↔ 서버



통신 2: 블루투스 (BLE) — 스마트폰 ↔ 보드



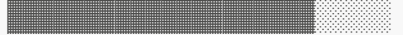
(블루투스, 가까이 있어야 함 - 약 10m 이내)
(인터넷 필요 없음)

두 통신의 차이

항목	인터넷 (HTTPS)	블루투스 (BLE)
연결 대상	스마트폰 ↔ 서버	스마트폰 ↔ 보드
거리	무제한	~10미터
속도	빠름	느림 (240바이트씩)
용도	AI 코드 생성 + 빌드	펌웨어 전송 + 직접 제어
필요한 것	Wi-Fi 또는 LTE	블루투스 켜기

9. 시간이 얼마나 걸리나요?

프롬프트를 입력하고 버튼을 누른 후, 보드가 동작하기까지:

① AI 코드 생성		~8초
② 빌드 (컴파일)		~25초
③ 다운로드		~1초
④ BLE 전송		~17초
합계: 약 35~70초 (프롬프트 난이도에 따라 다름)		

시간이 가장 오래 걸리는 곳:

- 1️⃣ 빌드 (컴파일) - 코드를 기계어로 번역하는 데 가장 오래 걸림
- 2️⃣ BLE 전송 - 650KB를 블루투스로 보내는 데 시간이 걸림
- 3️⃣ AI 코드 생성 - AI가 코드를 만드는 시간

10. 직접 제어 vs 프롬프트 제어

이 시스템에는 보드를 제어하는 **2가지 방법**이 있습니다.

방법 1: 프롬프트 제어 (🏠 홈 탭)

"학교종 멜로디 연주하고 LED 무지개로 깜빡여줘"

▼ (AI → 빌드 → BLE 전송 → 재부팅)

▼ 약 50초 후...

 보드가 영구적으로 새 프로그램을 실행

- **장점:** 복잡한 동작도 가능, 보드에 프로그램이 저장됨 (전원 꺼도 유지)
- **단점:** 시간이 걸림 (35~70초)
- **사용 시점:** 새로운 기능을 만들 때

방법 2: 직접 제어 (🎮 패드 탭)

[ 빨강 버튼 터치]

|

▼ (블루투스로 명령 전송)

|

▼ 즉시! (<0.1초)

 빨간 LED 켜짐

- **장점:** 즉시 반응, 실시간 제어
- **단점:** 단순한 동작만 가능 (LED 켜기/끄기, 부저 울리기 등)
- **사용 시점:** LED나 부저를 바로 시험하고 싶을 때

11. 자주 묻는 질문 (FAQ)

Q1. 인터넷이 없으면 사용할 수 없나요?

AI 코드 생성 + 빌드는 클라우드 서버에서 하므로 인터넷(Wi-Fi 또는 LTE)이 필요합니다. 단, 🎮 **패드 탭의 직접 제어**는 블루투스만 있으면 되므로 인터넷 없이도 사용 가능합니다.

Q2. 보드의 전원을 끄면 프로그램이 사라지나요?

아니요! **프롬프트로 만든 프로그램**은 보드의 메모리(Flash)에 저장됩니다. 전원을 껐다 켜도 마지막으로 전송한 프로그램이 자동으로 실행됩니다.

Q3. 한 보드를 여러 학생이 동시에 사용할 수 있나요?

아니요, 하나의 보드에는 **한 번에 하나의 블루투스 연결**만 가능합니다. 각 학생은 자기 보드를 사용합니다. 보드마다 이름이 다릅니다 (UTTEC-OTA, UTTEC-01 등).

Q4. 에러가 나면 어떻게 하나요?

AI가 만든 코드가 완벽하지 않을 수 있습니다. 이때 서버가 자동으로 **최대 3번까지 재시도**합니다. 그래도 안 되면 프롬프트를 더 구체적으로 고쳐서 다시 시도하면 됩니다.

 "멋진 거 만들어줘" → AI가 뭘 해야 할지 모름

 "빨간 LED 0.5초마다 깜빡여줘" → 명확해서 성공률 높음

Q5. OLED에 한글을 표시할 수 있나요?

현재는 **영어, 숫자, 특수문자**만 표시 가능합니다. 한글 폰트는 아직 지원하지 않습니다.

Q6. 위험하지 않나요?

ESP32 보드는 **3.3V 저전압**으로 동작하므로 감전 위험이 없습니다. USB 전원(5V)으로 구동되며, 만져도 안전합니다.

12. 용어 정리

용어	쉬운 설명
프롬프트 (Prompt)	AI에게 보내는 주문서. 한국어로 적는다
펌웨어 (Firmware)	보드 안에 들어가는 프로그램
빌드 (Build)	코드를 기계어로 번역하는 과정 (= 컴파일)
컴파일 (Compile)	빌드와 같은 뜻. 사람 언어 → 기계 언어
BLE (Bluetooth Low Energy)	저전력 블루투스. 보드와 무선 통신
OTA (Over The Air)	무선으로 프로그램을 보드에 전송하는 방식
GPIO (General Purpose I/O)	범용 입출력 핀. 각 핀에 번호가 있다 (예: 25번)
클라우드 (Cloud)	인터넷에 있는 서버 컴퓨터
AI (Artificial Intelligence)	인공지능. 여기서는 코드를 만들어주는 Claude
ESP32	우리가 사용하는 작은 컴퓨터 칩의 이름
Arduino	ESP32에 프로그램을 만드는 도구/방식의 이름
OLED	작은 디스플레이 화면 (128x64 점)
AHT20	온도와 습도를 측정하는 센서의 이름
I2C	부품끼리 대화하는 통신 방식 (OLED, 센서가 사용)
NVS	보드의 영구 저장소. 전원 꺼도 데이터 유지

13. 더 알고 싶다면

이 문서는 전체 흐름을 쉽게 설명한 것입니다. 더 자세한 기술 내용이 궁금하다면 아래 문서를 참고하세요.

문서	내용	난이도
시스템_프롬프트_설명서.md	AI에게 보내는 시스템 프롬프트의 구조와 규칙	★★★★
빌드_파이프라인_상세분석_및_최적화.md	빌드 과정의 시간 분석과 최적화 방안	★★★★★
웹앱_구조.md	웹앱의 HTML/JavaScript 구조	★★★★

문서	내용	난이도
UTTEC_바이브코딩_교육플랫폼_시스템사양서_v2.0.md	전체 시스템 기술 사양	★★★★★
예시_프롬프트_목록.md	125개 예시 프롬프트 모음	★

작성일: 2026-04-16 대상: 바이브코딩을 처음 접하는 학생, 교사, 학부모 기반: UTTEC 바이브코딩 교육 플랫폼 시스템 사양서 v2.0